

[KAN-1] P1 : Projet CampaignIQ Création: 18/mai/26 Mise à jour: 19/mai/26 Date d'échéance: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)

Type:	Epic	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 hzzzzz:
Start date:	18/mai/26

Description

NovaBrand est une entreprise active spécialisée dans la vente de produits grand public en ligne. Elle opère dans plusieurs pays européens et investit chaque mois des budgets significatifs dans des campagnes marketing multi-canaux. Son équipe data a besoin d'analyser la performance de ces campagnes pour orienter les décisions budgétaires du trimestre prochain. C'est vous qui jouez le rôle de l'équipe data. Vous allez d'abord générer un dataset réaliste qui simule les données brutes exportées depuis leur outil de gestion de campagnes, puis vous allez le nettoyer, le transformer et l'analyser avec pandas, avant de le charger dans permettre des requêtes SQL ultérieures.

P1 : Projet CampaignIQ (KAN-1)

 **[KAN-2] P1-T1 : Génération du dataset (faker)** Création: 18/mai/26 Mise à jour: 18/mai/26 Date d'échéance: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1 : Projet CampaignIQ

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	2 heures	Temps consacré:	2 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-8	Installer les bibliothèques	Subtask	Terminé	Naia Landaboure
	KAN-9	Créer le fichier generate_data.py	Subtask	Terminé	Naia Landaboure
	KAN-10	Introduire des problèmes de qualité i...	Subtask	Terminé	Naia Landaboure
Rank:	0 i00007:				
Start date:	18/mai/26				

Description
Générer le dataset avec Faker + numpy

P1 : Projet CampaignIQ (KAN-1)

 **[KAN-3] P1-T2 : Exploration du dataset** Création: 18/mai/26 Mise à jour: 18/mai/26 Date d'échéance: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1 : Projet CampaignIQ

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	1 heure	Estimation restante:	1 hour
Σ Temps consacré:	2 heures	Temps consacré:	2 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-11	Lire le CSV avec pandas	Subtask	Terminé	Naia Landaboure
	KAN-12	Explorer les données	Subtask	Terminé	Naia Landaboure
Rank:	0 i0000f:				
Start date:	18/mai/26				

Description
Lire le csv puis analyser le dataset

P1 : Projet CampaignIQ (KAN-1)

 **[KAN-4] P1-T3 : Nettoyage et transformation** Création: 18/mai/26 Mise à jour: 18/mai/26 Date d'échéance: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1 : Projet CampaignIQ

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	2 heures	Temps consacré:	2 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-13	Corriger le type de la colonne date	Subtask	Terminé	Naia Landaboure
	KAN-14	Corriger les valeurs manquantes	Subtask	Terminé	Naia Landaboure
	KAN-15	Supprimer les doublons	Subtask	Terminé	Naia Landaboure
	KAN-16	Corriger les valeurs aberrantes	Subtask	Terminé	Naia Landaboure
	KAN-17	Créer les nouvelles colonnes d'indica...	Subtask	Terminé	Naia Landaboure
	KAN-18	Créer les colonnes temporelles	Subtask	Terminé	Naia Landaboure

	KAN-19	Sauvegarder le dataset en campaign.csv	Subtask	Terminé	Naia Landaboure
Rank:	0 10000n:				
Start date:	18/mai/26				

Description
Nettoyer : valeurs manquantes, doublons, aberrations Calculer CTR, CAC, ROAS, CVR

<u>P1 : Projet CampaignIQ</u> (KAN-1)	
 [KAN-5] <u>P1-T4 : Agrégation et analyse</u> Création: 18/mai/26 Mise à jour: 19/mai/26 Date d'échéance: 19/mai/26 Résolu: 19/mai/26	
État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1 : Projet CampaignIQ

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	2 heures	Temps consacré:	2 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-20	Agréger par groupe	Subtask	Terminé	Naia Landaboure
	KAN-21	Créer un tableau croisé dynamique	Subtask	Terminé	Naia Landaboure
	KAN-22	Analyse temporelle	Subtask	Terminé	Naia Landaboure
	KAN-23	Créer le rapport final	Subtask	Terminé	Naia Landaboure
Rank:	0 i0000v:				
Start date:	19/mai/26				

Description
Agréger : groupby, pivot_table, resample

P1 : Projet CampaignIQ (KAN-1)

 [KAN-24] **P1-T5 : Export vers PostgreSQL** Création: 18/mai/26 Mise à jour: 19/mai/26 Date d'échéance: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1 : Projet CampaignIQ

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	1 heure	Temps consacré:	1 hour
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-25	Installation PostgreSQL	Subtask	Terminé	Naia Landaboure
	KAN-26	Créer le moteur de connexion	Subtask	Terminé	Naia Landaboure
	KAN-27	Charger le DdataFrame dans PostgreSQL	Subtask	Terminé	Naia Landaboure
	KAN-28	Vérification dans psql et depuis Python	Subtask	Terminé	Naia Landaboure
	KAN-29	Fermer le moteur	Subtask	Terminé	Naia Landaboure
Rank:	0 i0000z:				
Start date:	19/mai/26				

Description

Exporter le dataframe campaign_clean.csv sur postgresQL



[KAN-6] **P2 : Projet DataPulse** Création: 18/mai/26 Mise à jour: 20/mai/26 Date d'échéance: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)

Type:	Epic	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00013:
Start date:	19/mai/26

Description

NovaBrand veut enrichir ses données de campagnes avec des sources externes : météo au moment des campagnes, indicateurs économiques des pays cibles, et taux de change pour convertir les revenus. Ce projet vous apprend à collecter ces données automatiquement depuis le web et des APIs publiques, à les croiser avec le dataset NovaBrand, et à générer un rapport hebdomadaire automatisé.

[KAN-30] [P3 : Projet RetailsLens](#) Création: 18/mai/26 Mise à jour: 22/mai/26 Date d'échéance: 22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)

Type:	Epic	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00017:
Start date:	21/mai/26

Description

Ingérer 500 000 transactions e-commerce réelles, analyser avec des CTEs et window functions SQL, construire une analyse RFM et visualiser dans Power BI.

P2 : Projet DataPulse (KAN-6)



[KAN-7] P2-T1 : Scraping web

Création: 18/mai/26 Mise à jour: 19/mai/26 Date d'échéance: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2 : Projet DataPulse

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	3 heures	Temps consacré:	3 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-31	Récupérer du HTML	Subtask	Terminé	Naia Landaboure
	KAN-32	Analyser et extraire	Subtask	Terminé	Naia Landaboure
	KAN-33	Exercice quotes.toscrape.com	Subtask	Terminé	Naia Landaboure
	KAN-34	Stocker en CSV et PostgreSQL	Subtask	Terminé	Naia Landaboure
Rank:	0 i00018:				
Start date:	19/mai/26				

Description

Scraping web requests + BeautifulSoup <http://quotes.toscrape.com>

<u>P2 : Projet DataPulse</u> (KAN-6)	
 [KAN-35] <u>P2-T2 : Collecte des données en JSON</u> Création: 19/mai/26 Mise à jour: 20/mai/26 Date d'échéance: 20/mai/26 Résolu: 20/mai/26	
État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2 : Projet DataPulse

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	4 heures	Temps consacré:	4 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-37	API Taux de change	Subtask	Terminé	Naia Landaboure
	KAN-38	API Météo	Subtask	Terminé	Naia Landaboure
	KAN-39	API Pays	Subtask	Terminé	Naia Landaboure
	KAN-40	Merge des différentes APIs	Subtask	Terminé	Naia Landaboure
	KAN-41	Gérer les erreurs d'APIs	Subtask	Terminé	Naia Landaboure
Rank:	0 i00019:				
Start date:	20/mai/26				

Description
APIs : Open-Meteo REST Countries · Frankfurter Parsing JSON

P2 : Projet DataPulse (KAN-6)

 **[KAN-36] P2-T3 : Reporting automatisé** Création: 19/mai/26 Mise à jour: 20/mai/26 Date d'échéance: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2 : Projet DataPulse

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	4 heures	Temps consacré:	4 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-42	Croiser Novabrand avec les données APIS	Subtask	Terminé	Naia Landaboure
	KAN-43	Générer le CSV enrichi	Subtask	Terminé	Naia Landaboure
	KAN-44	Générer un rapport lisible	Subtask	Terminé	Naia Landaboure
	KAN-45	Exporter vers PostgreSQL	Subtask	Terminé	Naia Landaboure
Rank:	0 i0001f:				
Start date:	20/mai/26				

Description
Reporting automatisé CSV + rapport texte

P1-T1 : Génération du dataset (faker) (KAN-2)



[KAN-8] Installer les bibliothèques

Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T1 : Génération du dataset (faker)

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0001j:
--------------	-----------

Description

Ouvrez VSCode dans un nouveau dossier campaigniq/. Créez et activez votre environnement virtuel, puis installez les bibliothèques nécessaires.

```
C:\Users\VotreNom\Documents\campaigniq python -m venv
```

```
.venv .venv\Scripts\Activate.ps1
```

```
pip install pandas faker numpy sqlalchemy psycpg2-binary openpyxl
```

P1-T1 : Génération du dataset (faker) (KAN-2)

 **[KAN-9] Créer le fichier generate_data.py** Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T1 : Génération du dataset (faker)

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0001r:
--------------	-----------

Description

Créez un fichier generate_data.py dans votre dossier campaigniq/. Le script génère 5 000 lignes de données de campagnes marketing, avec des problèmes de qualité intentionnels pour l'exercice de nettoyage.

```
import pandas as pd
import numpy as np
from faker import Faker
import random
```

```
# — Initialisation —————
fake = Faker()
np.random.seed(42) # reproductibilité : même seed = mêmes données
random.seed(42)
N = 5000 # nombre de campagnes à générer
```

```
# — Listes de valeurs possibles —————

channels = ['email', 'social', 'search', 'display']
countries = ['France', 'Germany', 'Spain', 'Italy', 'Netherlands']
```

```

segments = ['new', 'returning', 'vip']

# — Génération des données de base —————
rows = []
for i in range(N):
    channel = random.choice(channels)
    country = random.choice(countries)
    segment = random.choice(segments)

    # Impressions selon le canal (search/display ont plus d'impressions)
    if channel in ['search', 'display']:
        impressions = int(np.random.lognormal(10.5, 0.8))
    else:
        impressions = int(np.random.lognormal(9.5, 0.7))

    # CTR (taux de clic) réaliste par canal
    ctr_base = {'email': 0.18, 'social': 0.04, 'search': 0.08, 'display':
0.002}
    ctr = max(0, np.random.normal(ctr_base[channel],
ctr_base[channel]*0.3))
    clicks = int(impressions * ctr)

    # Taux de conversion et spend
    conv_rate = max(0, np.random.normal(0.03, 0.01))
    conversions = int(clicks * conv_rate)
    spend = round(np.random.lognormal(7, 0.6), 2)
    revenue = round(conversions * np.random.uniform(80, 150), 2)
    rows.append({
        'campaign_id' : f'CAMP-{i+1:05d}',
        'date': fake.date_between(start_date='-2y', end_date='today'),
        'channel': channel,
        'campaign_name': f'{fake.catch_phrase()} {channel.title()}'
{country[:2].upper()}',
        'country': country,
        'segment': segment,
        'impressions' : impressions,
        'clicks': clicks,
        'conversions' : conversions,
        'spend': spend,
        'revenue': revenue,
    })

df = pd.DataFrame(rows)

```

P1-T1 : Génération du dataset (faker) (KAN-2)

 **[KAN-10] Introduire des problèmes de qualité intentionnels** Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T1 : Génération du dataset (faker)

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0001z:
--------------	-----------

Description

Simuler des valeurs manquantes, des doublons, des valeurs aberrantes, et des types de colonnes incorrects

A rajouter sur generate_data.py

```
# — Introduire des problèmes de qualité —————

# 1. Valeurs manquantes dans 'spend' et 'country' (~5% des lignes)
idx_spend = np.random.choice(df.index, size=int(N*0.05), replace=False)
idx_country = np.random.choice(df.index, size=int(N*0.04), replace=False)
df.loc[idx_spend, 'spend'] = np.nan
df.loc[idx_country, 'country'] = np.nan

# 2. Doublons — dupliquer 50 lignes aléatoires
duplicates = df.sample(50, random_state=42)
df = pd.concat([df, duplicates], ignore_index=True)

# 3. Valeurs aberrantes dans 'clicks' — quelques valeurs impossibles
idx_aberrant = np.random.choice(df.index, size=10, replace=False)
df.loc[idx_aberrant, 'clicks'] = -999 # clics négatifs → impossible

# 4. Type incorrect — date stockée comme chaîne dans quelques lignes
```

```
df['date'] = df['date'].astype(str) # tout en string pour simuler un
export CSV

# — Mélanger et sauvegarder —————

df = df.sample(frac=1, random_state=42).reset_index(drop=True)
df.to_csv('campaigns.csv', index=False, encoding='utf-8')

print(f"Dataset généré : {len(df)} lignes × {len(df.columns)} colonnes")
print(f"Fichier sauvegardé : campaigns.csv")
print(df.head(3))
```

P1-T2 : Exploration du dataset (KAN-3)

 [KAN-11] Lire le CSV avec pandas Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T2 : Exploration du dataset

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00027:
-------	-----------

Description

Créez un nouveau fichier analyse.py

```
import pandas as pd
import numpy as np

# Lire le CSV généré
df = pd.read_csv('campaigns.csv')
print("Dimensions :", df.shape)      # (5050, 11)
print(df.head())                     # 5 premières lignes
```

P1-T2 : Exploration du dataset (KAN-3)

 **[KAN-12] Explorer les données** Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T2 : Exploration du dataset

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0002f:
--------------	-----------

Description
<code>df.info()</code>
<code>df.describe ()</code>
<code>df.isnull().sum()</code>
<code>df.duplicated().sum()</code>

P1-T3 : Nettoyage et transformation (KAN-4)

 **[KAN-13] Corriger le type de la colonne date** Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0002n:
-------	-----------

Description

```
df['date'] = pd.to_datetime(df['date'])

# Vérification
print(df['date'].dtype)      # datetime64[ns]  ✓
print(df['date'].min())     # date la plus ancienne
print(df['date'].max())     # date la plus récente
```

P1-T3 : Nettoyage et transformation (KAN-4)



[KAN-14] Corriger les valeurs manquantes Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0002v:
--------------	-----------

Description

```
# — Stratégie 1 : supprimer les lignes où country est inconnu —
# On ne peut pas analyser les performances par pays sans le pays
print("Avant dropna, nb country non vides:", len(df), "lignes")
df = df.dropna(subset=['country'])
print("Après dropna, nb country non vides:", len(df), "lignes")

# — Stratégie 2 : remplacer spend manquant par la médiane —
# On ne veut pas perdre des lignes pour un budget manquant.
# La médiane est plus robuste que la moyenne face aux valeurs extrêmes.
mediane_spend = df['spend'].median()
df['spend'] = df['spend'].fillna(mediane_spend)
print(f"spend NaN remplacés par la médiane : {mediane_spend:.2f} €")

# Vérification - plus aucun NaN
print(df[['spend', 'country']].isnull().sum())
```

P1-T3 : Nettoyage et transformation (KAN-4)



[KAN-15] Supprimer les doublons Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00033:
-------	-----------

Description

```
print("Avant dédoublonnage :", len(df), "lignes")
df = df.drop_duplicates()
df = df.reset_index(drop=True)    # réindexer proprement après suppression
print("Après dédoublonnage :", len(df), "lignes")

# Vérification
print("Doublons restants :", df.duplicated().sum())    # doit afficher 0
```

P1-T3 : Nettoyage et transformation (KAN-4)



[KAN-16] Corriger les valeurs aberrantes Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0003b:
--------------	-----------

Description

```
# Compter les lignes aberrantes avant filtrage
# 10 lignes
print("Clics négatifs :", (df['clicks'] < 0).sum())
print("Clics > impressions :", (df['clicks'] > df['impressions']).sum())

# Filtrer : garder seulement les lignes valides
df = df[df['clicks'] >= 0]
df = df[df['clicks'] <= df['impressions']]
df = df.reset_index(drop=True)
print("Après nettoyage aberrations :", len(df), "lignes")
```

P1-T3 : Nettoyage et transformation (KAN-4)



[KAN-17] Créer les nouvelles colonnes d'indicateurs Création: 18/mai/26 Mise à jour:

18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0003j:
--------------	-----------

Description

```
# CTR (clic / impressions) : % de personnes qui ont cliqué
# éviter la division par zéro avec np.where
# np.where(condition, valeur_si_vrai, valeur_si_faux)
df['ctr'] = np.where(
    df['impressions'] > 0,
    df['clicks'] / df['impressions'],
    0
)

# CAC - coût d'acquisition (spend / conversions) : Combien coûte chaque
client acquis. Plus c'est bas, mieux c'est.
# (NaN si 0 conversions - normal, on garde)
df['cac'] = df['spend'] / df['conversions'].replace(0, np.nan)

# ROAS - retour sur investissement publicitaire (revenue / spend) :
Combien rapporte chaque euro dépensé. ROAS > 1 = rentable.
df['roas'] = df['revenue'] / df['spend'].replace(0, np.nan)

# CVR - taux de conversion (conversions / clicks) : % de cliqueurs qui
achètent.
df['cvr'] = np.where(
    df['clicks'] > 0,
    df['conversions'] / df['clicks'],
```

```
    0
)
# Vérification des nouvelles colonnes
print(df[['ctr', 'cac', 'roas', 'cvr']].describe().round(3))
```

P1-T3 : Nettoyage et transformation (KAN-4)

 **[KAN-18] Créer les colonnes temporelles** Création: 18/mai/26 Mise à jour: 18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0003r:
--------------	-----------

Description
<pre>df['year'] = df['date'].dt.year df['month'] = df['date'].dt.month df['week'] = df['date'].dt.isocalendar().week.astype(int) df['day_of_week'] = df['date'].dt.day_name() # 'Monday', 'Tuesday', etc.</pre>

P1-T3 : Nettoyage et transformation (KAN-4)



[KAN-19] Sauvegarder le dataset en campaign.csv Création: 18/mai/26 Mise à jour:

18/mai/26 Résolu: 18/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T3 : Nettoyage et transformation

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0003z:
--------------	-----------

Description

```
df.to_csv('campaigns_clean.csv', index=False)
print(f"Dataset nettoyé : {len(df)} lignes, {len(df.columns)} colonnes")
print("Colonnes :", df.columns.tolist())
```

P1-T4 : Agrégation et analyse (KAN-5)



[KAN-20] Agréger par groupe

Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T4 : Agrégation et analyse

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00047:
-------	-----------

Description

```
# — Agrégation simple : performance par canal —————

perf_canal = df.groupby('channel').agg(
    nb_campagnes    = ('campaign_id', 'count'),
    spend_total     = ('spend', 'sum'),
    revenue_total   = ('revenue', 'sum'),
    conversions     = ('conversions', 'sum'),
    ctr_moyen       = ('ctr', 'mean'),
    roas_moyen      = ('roas', 'mean')
).round(2)

print(perf_canal.sort_values('roas_moyen', ascending=False))

# — Agrégation multi-niveaux : canal × pays —————

# On regroupe sur deux colonnes simultanément
perf_canal_pays = df.groupby(['channel', 'country']).agg(
    spend_total     = ('spend', 'sum'),
    revenue_total   = ('revenue', 'sum'),
    roas_moyen      = ('roas', 'mean')
).round(2)

# Trouver le meilleur ROAS par pays (indépendamment du canal)
```

```
meilleur_roas_pays =  
df.groupby('country')['roas'].mean().sort_values(ascending=False)  
print(meilleur_roas_pays)  
  
#meilleur roas pays : Allemagne, et Italie  
  
# — Top 5 campagnes par revenue —————  
cols = ['campaign_name', 'channel', 'country', 'revenue', 'roas']  
top5 = df.nlargest(5, 'revenue')[cols]  
print(top5)
```

P1-T4 : Agrégation et analyse (KAN-5)

 **[KAN-21] Créer un tableau croisé dynamique** Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T4 : Agrégation et analyse

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0004f:
--------------	-----------

Description

```
# Tableau croisé : canal (lignes) × segment (colonnes) → ROAS moyen
tableau_roas = pd.pivot_table(
    df,
    values='roas',
    index='channel',
    columns='segment',
    aggfunc='mean'
).round(2)
print(tableau_roas)

# Spend total par canal × pays
tableau_spend = pd.pivot_table(
    df,
    values='spend',
    index='channel',
    columns='country',
    aggfunc='sum',
    margins=True,
    margins_name='Total'
).round(0)

print(tableau_spend)
```



P1-T4 : Agrégation et analyse (KAN-5)



[KAN-22] Analyse temporelle Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T4 : Agrégation et analyse

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0004n:
-------	-----------

Description

```
# Définir la date comme index pour resample
df_ts = df.set_index('date').sort_index()

# — Revenus par semaine —————
revenus_semaine = df_ts['revenue'].resample('W').sum()
print(revenus_semaine.tail(8))          # 8 dernières semaines
#affiche le jour de fin de semaine (dimanche) et le total des revenus pour
la semaine écoulée.

# — Spend et revenue par mois —————
mensuel = df_ts[['spend', 'revenue', 'conversions']].resample('ME').sum()
# 'ME' = Month End – dernier jour du mois (pandas
# Ancienne syntaxe : 'M' (toujours valide)
# — Calculer le ROAS mensuel depuis les totaux mensuels

mensuel['roas_mensuel'] = (mensuel['revenue'] / mensuel['spend']).round(2)
print(mensuel.tail(6))                  # 6 derniers mois
#affiche le dernier jour du mois et le total du spend, revenue,
conversions et le roas_mensuel pour ce mois.

# — Croiser resample et groupby —————
# Revenus mensuels par canal – un peu plus avancé
mensuel_canal = (
    df.groupby([pd.Grouper(key='date', freq='ME'), 'channel'])
```

```
    ['revenue'].sum().unstack()  
)  
print(mensuel_canal.tail(3))
```

P1-T4 : Agrégation et analyse (KAN-5)



[KAN-23] Créer le rapport final

Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T4 : Agrégation et analyse

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0004v:
-------	-----------

Description

```
print("="*50)
print("RAPPORT CAMPAIGNIQ - SYNTHÈSE")
print("="*50)
print(f"Période analysée : {df['date'].min().date()} → {df['date'].max().date()}")
print(f"Nombre de campagnes : {len(df):,}")
print(f"Spend total : {df['spend'].sum():,.0f} €")
print(f"Revenue total : {df['revenue'].sum():,.0f} €")
print(f"ROAS global : {df['revenue'].sum()/df['spend'].sum():.2f}")
print(f"Total conversions : {df['conversions'].sum():,}")
print(f"CAC moyen : {df['cac'].median():.2f} €")
print(f"Meilleur canal (ROAS) : {df.groupby('channel')['roas'].mean().idxmax()}")
print(f"Meilleur pays (revenue): {df.groupby('country')['revenue'].sum().idxmax()}")
```

P1-T5 : Export vers PostgreSQL (KAN-24)

 **[KAN-25] Installation PostgreSQL** Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T5 : Export vers PostgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00053:
--------------	-----------

Description

Dans la fenetre psql

```
CREATE DATABASE campaigniq;
```

```
\c campaigniq
```

P1-T5 : Export vers PostgreSQL (KAN-24)



[KAN-26] Créer le moteur de connexion Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T5 : Export vers PostgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005b:
--------------	-----------

Description

Dans VS

```
from sqlalchemy import create_engine, text
import pandas as pd
import numpy as np

# Lire le CSV généré
df = pd.read_csv('campaigns_clean.csv', parse_dates=['date'])

# — Créer le moteur de connexion à PostgreSQL Portable —
# Format de l'URL : dialect+driver: user:password@host:port/database
engine =
create_engine("postgresql+psycopg2://postgres:@localhost:5432/campaigniq")

# —————
#dialect      user      pwd      hôte      port base

# Tester la connexion
try:
    with engine.connect() as conn:
        result = conn.execute(text("SELECT version();"))
```

```
        print("Connexion OK -", result.fetchone()[0])
except Exception as e:
    print("Erreur de connexion :", e)
```

P1-T5 : Export vers PostgreSQL (KAN-24)



[KAN-27] Charger le DDataFrame dans PostgreSQL Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T5 : Export vers PostgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005j:
--------------	-----------

Description

```
df.to_sql(  
    name='campaigns',      # nom de la table à créer  
    con=engine,            # moteur SQLAlchemy  
    if_exists='replace',   # 'replace' : supprimer et recréer si existe  
    déjà  
    index=False,          # ne pas écrire l'index pandas comme colonne  
    chunksize=500,        # insérer par lots de 500 lignes (plus stable)  
    method='multi'        # insertion multi-valeurs – plus rapide  
)  
print("Table 'campaigns' chargée dans PostgreSQL )
```

P1-T5 : Export vers PostgreSQL (KAN-24)



[KAN-28] Vérification dans psql et depuis Python Création: 18/mai/26 Mise à jour:

19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P1-T5 : Export vers PostgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005r:
--------------	-----------

Description

Dans psql

```
campaigniq=# SELECT COUNT(*) FROM campaigns;
```

```
campaigniq=# SELECT channel, COUNT(*), ROUND(AVG(roas)::numeric,2) AS  
roas_moyen  
campaigniq-# FROM campaigns  
campaigniq-# GROUP BY channel  
campaigniq-# ORDER BY roas_moyen DESC;
```

Dans VS

```
sql = (  
    "SELECT channel, COUNT(*) as nb,"  
    "ROUND(AVG(roas) ::numeric, 2) as roas_moyen "  
    "FROM campaigns GROUP BY channel ORDER BY roas_moyen DESC"  
)  
df_check = pd.read_sql(sql, con=engine)  
print(df_check)
```

```
# Compter le nombre total de lignes
nb_lignes = pd.read_sql("SELECT COUNT(*) as total FROM campaigns",
con=engine)
print(f"Total lignes en base : {nb_lignes['total'][0]}")
```

<u>P1-T5 : Export vers PostgreSQL</u> (KAN-24)			
 [KAN-29] <u>Fermer le moteur</u> Création: 18/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26			
État:	Terminé		
Projet:	Workshop		
Composants:	Aucun(e)		
Versions affectées:	Aucun(e)		
Versions corrigées:	Aucun(e)		
Parent:	P1-T5 : Export vers PostgreSQL		

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005z:
--------------	-----------

Description
<pre>engine.dispose() print("Connexion fermée.")</pre>

P2-T1 : Scraping web (KAN-7)

 [KAN-31] Récupérer du HTML Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T1 : Scraping web

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005z:i
-------	------------

Description

```
import requests

url = "http://quotes.toscrape.com/"
response = requests.get(url)

# 200 = succès, 404 = introuvable, 403 = accès refusé
print(response.status_code)
print(response.text[:500])
# les 500 premiers chars du HTML
```

P2-T1 : Scraping web (KAN-7)



[KAN-32] Analyser et extraire

Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T1 : Scraping web

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005z:r
--------------	------------

Description

```
from bs4 import BeautifulSoup
import requests

url = "http://quotes.toscrape.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text)          # "Quotes to Scrape"
quotes = soup.find_all("div", class_="quote")
print(f"{{len(quotes)}} citations sur cette page") # 10

premier = quotes[0]
print(premier.find("span", class_="text").text)
print(premier.find("small", class_="author").text)
```

P2-T1 : Scraping web (KAN-7)



[KAN-33] [Exercice quotes.toscrape.com](#) Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T1 : Scraping web

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0005z:v
-------	------------

Description

```
import requests
import pandas as pd
from bs4 import BeautifulSoup
import time
from sqlalchemy import create_engine

base_url = "http://quotes.toscrape.com"
all_quotes = []
page = 1

while True:
    url = f"{base_url}/page/{page}/"
    response = requests.get(url)
    if response.status_code != 200:
        break
    # Si ce n'est pas = à , ce n'est pas la bonne page, on arrête la boucle
    soup = BeautifulSoup(response.text, "html.parser")
    quote_divs = soup.find_all("div", class_="quote")
    if not quote_divs:
        break
    #quote_divs est une liste toutes les divs dont la classe est "quote".
    #Si cette liste est vide, cela signifie qu'il n'y a pas de citations sur
    la page, donc on arrête la boucle.
```

```
for div in quote_divs:
    texte = div.find("span", class_="text").text.strip(" ")
    auteur = div.find("small", class_="author").text.strip()
    tags = [t.text for t in div.find_all("a", class_="tag")]
    all_quotes.append({"page": page, "texte": texte, "auteur": auteur,
"tags": ", ".join(tags)})
#On extrait le texte de la citation, le nom de l'auteur et les tags
associés à chaque citation, puis on les stocke dans une liste de
dictionnaires appelée all_quotes.

print(f"Page {page} : {len(quote_divs)} citations")
page += 1
time.sleep(0.5)
```

P2-T1 : Scraping web (KAN-7)

 **[KAN-34]** Stocker en CSV et PostgreSQL Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T1 : Scraping web

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00060:
--------------	-----------

Description

```
engine =  
create_engine("postgresql+psycopg2://postgres:@localhost:5432/campaigniq")  
df_quotes.to_sql("quotes", con=engine, if_exists="replace", index=False)  
print("Table 'quotes' chargée )  
engine.dispose()
```

<u>P2-T2 : Collecte des données en JSON</u> (KAN-35)	
🔗[KAN-37] <u>API Taux de change</u> Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26	
État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T2 : Collecte des données en JSON

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00068:
--------------	-----------

Description
<pre> import requests, pandas as pd url = "https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD,GBP,CHF,JPY" # = "https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD,GBP,CHF,JPY" data = requests.get(url).json() print(f"Date des taux : {data['date']}") print(data["rates"]) df_taux = pd.DataFrame([{"date": data["date"], **data["rates"]})) df_taux.to_csv("taux_change.csv", index=False) # Taux historique – comparaison sur 1 an url_h = "https://api.frankfurter.dev/v1/2024-01-15?base=EUR&symbols=USD" # = "https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD&date=2024-01-15" data_h = requests.get(url_h).json() taux_actuel = data["rates"]["USD"] taux_ancien = data_h["rates"]["USD"] variation = ((taux_actuel - taux_ancien) / taux_ancien) * 100 print(f"Variation EUR/USD sur 1 an : {variation:+.2f}%") </pre>

P2-T2 : Collecte des données en JSON (KAN-35)



[KAN-38] API Météo

Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T2 : Collecte des données en JSON

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0006g:
-------	-----------

Description

```
import requests
import pandas as pd

capitales = {
    "France" : (48.8566, 2.3522),
    "Germany" : (52.5200, 13.4050),
    "Spain" : (40.4168, -3.7038),
    "Italy" : (41.9028, 12.4964),
    "Netherlands": (52.3676, 4.9041),
}

rows = []
for pays, (lat, lon) in capitales.items():
    url = (
        f"https://api.open-meteo.com/v1/forecast"
        f"?latitude={lat}&longitude={lon}"
        f"&current=temperature_2m,precipitation"
    )
    r = requests.get(url).json()
    rows.append({
        "country" : pays,
        "temperature_c" : r["current"]["temperature_2m"],
        "precipitation_mm" : r["current"]["precipitation"],
    })
```

```
print(f"    {pays} : {r['current']['temperature_2m']}°C")
```

```
df_meteo = pd.DataFrame(rows)  
df_meteo.to_csv("meteo.csv", index=False)
```

P2-T2 : Collecte des données en JSON (KAN-35)



[KAN-39] [API Pays](#) Création: 19/mai/26 Mise à jour: 19/mai/26 Résolu: 19/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T2 : Collecte des données en JSON

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0006o:
-------	-----------

Description

```
import requests, pandas as pd

pays_novabrand = ["France", "Germany", "Spain", "Italy", "Netherlands"]
rows = []

for pays in pays_novabrand:
    url = (f"https://restcountries.com/v3.1/name/{pays}"
           f"?fields=name,population,area,currencies,languages")
    data = requests.get(url).json()[0]
    rows.append({
        "country" : pays,
        "population": data["population"],
        "area_km2" : data["area"],
        "currency" : ", ".join(data["currencies"].keys()),
        "languages" : ", ".join(data["languages"].values()),
    })

df_countries = pd.DataFrame(rows)
df_countries.to_csv("countries.csv", index=False)
print(df_countries)
```

P2-T2 : Collecte des données en JSON (KAN-35)



[KAN-40] Merge des différentes APIs Création: 19/mai/26 Mise à jour: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T2 : Collecte des données en JSON

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0006w:
--------------	-----------

Description

```
import pandas as pd
import requests

df_meteo = pd.read_csv("meteo.csv")
df_countries = pd.read_csv("countries.csv")
df_taux = pd.read_csv("taux_change.csv")

# Jointure météo + pays sur la colonne "country"
df_ref = df_meteo.merge(df_countries, on="country", how="left")

# Ajouter le taux EUR/USD (même valeur pour tous les pays)
df_ref["eur_usd_rate"] = df_taux["USD"].iloc[0]
df_ref["pop_density"] = (df_ref["population"] /
df_ref["area_km2"]).round(1)

print(df_ref[["country", "temperature_c", "population", "eur_usd_rate"]])
df_ref.to_csv("pays_reference.csv", index=False)
print("✅ pays_reference.csv créé")
```

P2-T2 : Collecte des données en JSON (KAN-35)



[KAN-41] Gérer les erreurs d'APIs Création: 19/mai/26 Mise à jour: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T2 : Collecte des données en JSON

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00074:
--------------	-----------

Description

```
import time
import requests

def get_api(url, max_retries=3):

    for tentative in range(max_retries):
        try:
            r = requests.get(url, timeout=10)
            r.raise_for_status()
            return r.json()
        except requests.exceptions.Timeout:
            print(f"Timeout - tentative {tentative+1}/{max_retries}")
            time.sleep(2)
        except requests.exceptions.HTTPError as e:
            print(f"Erreur HTTP {e.response.status_code}")
            return None
    return None
```


P2-T3 : Reporting automatisé (KAN-36)



[KAN-43] Générer le CSV enrichi

Création: 20/mai/26 Mise à jour: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T3 : Reporting automatisé

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0007k:
--------------	-----------

Description

```
import pandas as pd
import requests
from datetime import date
from sqlalchemy import create_engine

# 1. Charger le dataset NovaBrand
df = pd.read_csv("campaigns_clean.csv")
df["date"] = pd.to_datetime(df["date"])

# 2. Agréger par pays - 7 derniers jours
date_limite = df["date"].max() - pd.Timedelta(days=7)
df_s = df[df["date"] >= date_limite]
kpi_pays = df_s.groupby("country").agg(
    nb_campagnes = ("campaign_id", "count"),
    spend_eur    = ("spend", "sum"),
    revenue_eur  = ("revenue", "sum"),
    conversions  = ("conversions", "sum"),
    roas_moyen   = ("roas", "mean")
).round(2).reset_index()

# 3. Taux de change EUR/USD
```

```
taux_data = requests.get("https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD").  
# "https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD"  
taux_usd = taux_data["rates"]["USD"]  
kpi_pays["spend_usd"] = (kpi_pays["spend_eur"] * taux_usd).round(2)  
kpi_pays["revenue_usd"] = (kpi_pays["revenue_eur"] * taux_usd).round(2)  
  
# 4. Météo actuelle par capitale  
capitales = {  
    "France": (48.8566, 2.3522), "Germany": (52.52, 13.405),  
    "Spain": (40.4168, -3.7038), "Italy": (41.9028, 12.4964),  
    "Netherlands": (52.3676, 4.9041),  
}  
meteo_rows = []  
for pays, (lat, lon) in capitales.items():  
    r = requests.get(f"https://api.open-  
meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m,precipitati  
    meteo_rows.append({"country": pays,  
        "temperature_c": r["current"]["temperature_2m"],  
        "precipitation_mm": r["current"]["precipitation"]})  
df_meteo = pd.DataFrame(meteo_rows)  
  
# 5. Merger tout ensemble  
rapport = kpi_pays.merge(df_meteo, on="country", how="left")  
rapport["taux_eur_usd"] = taux_usd  
rapport["date_rapport"] = str(date.today())  
rapport = rapport.sort_values("revenue_eur", ascending=False)  
print(rapport)
```

P2-T3 : Reporting automatisé (KAN-36)

 [KAN-44] Générer un rapport lisible Création: 20/mai/26 Mise à jour: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T3 : Reporting automatisé

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0007s:
-------	-----------

Description

```
aujourd_hui = date.today().strftime("%d/%m/%Y")

lignes = [
    f"=====",
    f"  RAPPORT NOVABRAND - {aujourd_hui}",
    f"=====",
    "",
    f"  Taux EUR/USD : {taux_usd} (BCE)",
    f"  Campagnes    : {len(df_s)}",
    "",
    f"  {'PAYS':<14} {'SPEND €':>9} {'REVENUE €':>11} {'ROAS':>6} {'T°':>5}",
    f"    {'-'*48}",
]

for _, row in rapport.iterrows():
    lignes.append(
        f"  {row['country']:<14} {row['spend_eur']:>9,.0f}"
        f"  {row['revenue_eur']:>11,.0f} {row['roas_moyen']:>6.2f}"
        f"  {row['temperature_c']:>4.1f} °C"
    )
lignes += [
    f"    {'-'*48}",
    f"  TOTAL SPEND    : {rapport['spend_eur'].sum():,.0f} EUR",
```

```
f"  TOTAL REVENUE : {rapport['revenue_eur'].sum():,.0f} EUR",  
f"  ROAS GLOBAL   :  
{rapport['revenue_eur'].sum()/rapport['spend_eur'].sum():.2f}",  
]
```

P2-T3 : Reporting automatisé (KAN-36)

 [KAN-45] Exporter vers PostgreSQL Création: 20/mai/26 Mise à jour: 20/mai/26 Résolu: 20/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P2-T3 : Reporting automatisé

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00080:
-------	-----------

Description

```
engine =  
create_engine("postgresql+psycopg2://postgres:@localhost:5432/campaigniq")  
rapport.to_sql("rapport_hebdo", con=engine, if_exists="replace",  
index=False)  
print("✅ Table rapport_hebdo chargée")  
engine.dispose()
```

P3 : Projet RetailsLens (KAN-30)



[KAN-46] P3-T1 : Ingestion Python -> PstgreSQL Création: 20/mai/26 Mise à jour:

21/mai/26 Date d'échéance: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3 : Projet RetailsLens

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	4 heures	Temps consacré:	4 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-50	Lire le fichier Excel avec pandas	Subtask	Terminé	Naia Landaboure
	KAN-51	Explorer et nettoyer les données	Subtask	Terminé	Naia Landaboure
	KAN-52	Charger dans PostgreSQL	Subtask	Terminé	Naia Landaboure
	KAN-53	Vérification dans psql	Subtask	Terminé	Naia Landaboure
Rank:	0 i00082:				
Start date:	21/mai/26				

Description

Lire le fichier onlie_retail_II.xlsx sur python et le charger dans postgresQL

P3 : Projet RetailLens (KAN-30)



[KAN-47] P3-T2 : SQL analytique - CTEs et windows functions Création:

20/mai/26 Mise à jour: 21/mai/26 Date d'échéance: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3 : Projet RetailLens

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	3 heures	Temps consacré:	3 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-54	Agrégations et group by	Subtask	Terminé	Naia Landaboure
	KAN-55	CTEs - Common Table Expressions	Subtask	Terminé	Naia Landaboure
	KAN-56	Windows functions	Subtask	Terminé	Naia Landaboure
	KAN-57	Exercices	Subtask	Terminé	Naia Landaboure
Rank:	0 i00084:				
Start date:	21/mai/26				

Description

Analyser le fichier retail via des requêtes sql sur Postgresql

P3 : Projet RetailsLens (KAN-30)

 **[KAN-48] P3-T3 : Analyse RFM - Segmentation client** Création: 20/mai/26 Mise à jour: 22/mai/26 Date d'échéance: 22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3 : Projet RetailsLens

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	3 heures	Temps consacré:	3 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Pièces jointes:	 image-20260521-130805.png				
Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-58	Calculer la Récence, Fréquence, Monta...	Subtask	Terminé	Naia Landaboure
	KAN-59	Scorer et segmenter les clients	Subtask	Terminé	Naia Landaboure
	KAN-60	Sauvegarder la table RFM pour Power BI	Subtask	Terminé	Naia Landaboure
Rank:	0 i0008c:				
Start date:	22/mai/26				

Description

La RFM est une méthode classique de segmentation client utilisée en marketing depuis les années 1990. Elle classe chaque client selon trois dimensions :

Une fois R, F et M calculés pour chaque client, on attribue un score de 1 à 5 sur chaque dimension (5 = meilleur), puis on combine les scores pour créer des segments clients : Champions, Clients dèles, Clients à risque, Clients perdus, etc

<u>P3 : Projet RetailsLens</u> (KAN-30)	
 [KAN-49] <u>P3-T4 : Visualisation sur Power BI</u> Création: 20/mai/26 Mise à jour: 22/mai/26 Date d'échéance: 22/mai/26 Résolu: 22/mai/26	
État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3 : Projet RetailsLens

Type:	Tâche	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Σ Estimation restante:	0 minutes	Estimation restante:	0 minutes
Σ Temps consacré:	4 heures	Temps consacré:	4 hours
Σ Estimation originale:	Non spécifié	Estimation originale:	Non spécifié

Sous-tâches:	Clé	Résumé	Type	État	Personne assignée
	KAN-61	Connecter Power BI à PostgreSQL	Subtask	Terminé	Naia Landaboure
	KAN-62	Crée les visuels - KPI cards	Subtask	Terminé	Naia Landaboure
	KAN-63	Graphiques et slicers	Subtask	Terminé	Naia Landaboure
Rank:	0 i0008g:				
Start date:	22/mai/26				

Description
Générer un daschboard

P3-T1 : Ingestion Python -> PstgreSQL (KAN-46)

 **[KAN-50] Lire le fichier Excel avec pandas** Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T1 : Ingestion Python -> PstgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0008o:
--------------	-----------

Description

```
import pandas as pd
import numpy as np

# Lire les deux feuilles du fichier Excel
print("Chargement du fichier Excel ")
df_2009 = pd.read_excel("online_retail_II.xlsx", sheet_name="Year 2009-2010")
df_2010 = pd.read_excel("online_retail_II.xlsx", sheet_name="Year 2010-2011")

# Concaténer les deux feuilles
df = pd.concat([df_2009, df_2010], ignore_index=True)
print(f"Dataset chargé : {len(df):,} lignes × {len(df.columns)} colonnes")

# Renommer les colonnes - noms propres en minuscule
df.columns = ["invoice", "stock_code", "description",
              "quantity", "invoice_date", "unit_price",
              "customer_id", "country"]

# Aperçu rapide
print(df.head(3))
```

```
print(df.dtypes)
print(df.isnull().sum())
```

P3-T1 : Ingestion Python -> PstgreSQL (KAN-46)

 **[KAN-51] Explorer et nettoyer les données** Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T1 : Ingestion Python -> PstgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0008w:
--------------	-----------

Description

```
# — Statistiques avant nettoyage —————
print(f"Lignes totales : {len(df):,}")
print(f"CustomerID manquants : {df['customer_id'].isnull().sum():,}")
print(f"Annulations (C) : {df['invoice'].astype(str).str.startswith('C').sum():,}")
print(f"Annulations sans CustomerID : {(df['invoice'].astype(str).str.startswith('C')) & (df['customer_id'].isnull())}.sum():,}")
print(f"Quantity <= 0 : {(df['quantity'] <= 0).sum():,}")
print(f"UnitPrice <= 0 : {(df['unit_price'] <= 0).sum():,}")

# — Nettoyage —————
# 1. Supprimer les lignes sans customer_id (transactions anonymes)
df = df.dropna(subset=["customer_id"])
print(f"\nAprès suppression des lignes sans customer_id : {len(df):,} lignes")

# 2. Supprimer les annulations (on ne veut que les ventes réelles)
df = df[~df["invoice"].astype(str).str.startswith("C")]
print(f"\nAprès suppression des annulations : {len(df):,} lignes")

# 3. Garder seulement quantity > 0 et unit_price > 0
df = df[(df["quantity"] > 0) & (df["unit_price"] > 0)]
```

```
print(f"\nAprès filtrage des valeurs négatives : {len(df):,} lignes")

# 4. Convertir customer_id en entier
df["customer_id"] = df["customer_id"].astype(int)

# 5. Calculer le montant total par ligne
df["total_price"] = (df["quantity"] * df["unit_price"]).round(2)
df = df.reset_index(drop=True)
print(f"\nAprès nettoyage : {len(df):,} lignes")
```

P3-T1 : Ingestion Python -> PstgreSQL (KAN-46)



[KAN-52] Charger dans PostgreSQL Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T1 : Ingestion Python -> PstgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i00094:
-------	-----------

Description

```
from sqlalchemy import create_engine
engine =
create_engine("postgresql+psycopg2://postgres:@localhost:5432/campaigniq"
)
print("Chargement dans PostgreSQL ")
df.to_sql(
    name="retail",
    con=engine,
    if_exists="replace",
    index=False,
    chunksize=1000,    # insérer 1 000 lignes à la fois
    method="multi" # insertion multi-valeurs – plus rapide
)

print("✅ Table 'retail' chargée dans PostgreSQL")
engine.dispose()
```

P3-T1 : Ingestion Python -> PstgreSQL (KAN-46)

 **[KAN-53] Vérification dans psql** Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T1 : Ingestion Python -> PstgreSQL

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i0009c:
--------------	-----------

Description

Compter les lignes SELECT COUNT FROM retail;

Aperçu des données SELECT * FROM retail LIMIT 5;

Vérifier la plage de dates SELECT MIN(invoice_date), MAX(invoice_date) FROM retail;

Nombre de clients uniques SELECT COUNT(DISTINCT customer_id) FROM retail;

Nombre de pays SELECT COUNT(DISTINCT country) FROM retail;

P3-T2 : SQL analytique - CTEs et windows functions (KAN-47)

 **[KAN-54] Agrégations et group by** Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T2 : SQL analytique - CTEs et windows functions

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Pièces jointes:	 image-20260521-130349.png
Rank:	0 i0009k:

Description



P3-T2 : SQL analytique - CTEs et windows functions (KAN-47)

 **[KAN-55] CTEs - Common Table Expressions** Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T2 : SQL analytique - CTEs et windows functions

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Pièces jointes:	 image-20260521-130436.png
Rank:	0 i0009s:

Description

P3-T2 : SQL analytique - CTEs et windows functions (KAN-47)

 **[KAN-56] Windows functions** Création: 21/mai/26 Mise à jour: 22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T2 : SQL analytique - CTEs et windows functions

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Pièces jointes:	 image-20260521-130456.png
Rank:	0 i000a0:

Description

P3-T2 : SQL analytique - CTEs et windows functions (KAN-47)

 [KAN-57] Exercices Création: 21/mai/26 Mise à jour: 21/mai/26 Résolu: 21/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T2 : SQL analytique - CTEs et windows functions

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Pièces jointes:	 image-20260521-130528.png  image-20260521-130536.png
Rank:	0 i000a8:

Description

P3-T3 : Analyse RFM - Segmentation client (KAN-48)

 **[KAN-58] Calculer la Récence, Fréquence, Montant en SQL** Création: 21/mai/26 Mise à jour: 22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T3 : Analyse RFM - Segmentation client

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i000ag:
--------------	-----------

Description

```
campaigniq=# WITH date_ref AS (SELECT MAX(invoice_date)::date AS ref FROM retail),
campaigniq=# rfm_raw AS (SELECT
campaigniq(# r.customer_id,
campaigniq(# (d.ref - MAX(r.invoice_date)::date) AS recence_jours,
campaigniq(# COUNT(DISTINCT r.invoice) AS frequence,
campaigniq(# ROUND(SUM(r.total_price)::numeric,2) AS montant
campaigniq(# FROM retail r
campaigniq(# CROSS JOIN date_ref d
campaigniq(# GROUP BY r.customer_id, d.ref)
campaigniq=# SELECT * FROM rfm_raw
campaigniq=# ORDER BY montant DESC LIMIT 10;
```

P3-T3 : Analyse RFM - Segmentation client (KAN-48)

 **[KAN-59] Scorer et segmenter les clients** Création: 21/mai/26 Mise à jour: 22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T3 : Analyse RFM - Segmentation client

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i000ao:
--------------	-----------

Description

```
campaigniq=# WITH date_ref AS (SELECT MAX(invoice_date)::date AS ref FROM retail),
campaigniq=# rfm_raw AS (SELECT
campaigniq(# customer_id,
campaigniq(# (MAX (d.ref) - MAX (invoice_date)::date) AS recence_jours,
campaigniq(# COUNT(DISTINCT invoice) AS frequence,
campaigniq(# ROUND(SUM(total_price)::numeric,2) AS montant
campaigniq(# FROM retail, date_ref d GROUP BY customer_id),
campaigniq=# rfm_scores AS (SÉLECT *,
campaigniq(# NTILE(5) OVER (ORDER BY recence_jours DESC) AS score_r,
campaigniq(# NTILE(5) OVER (ORDER BY frequence ASC) AS score_f,
campaigniq(# NTILE(5) OVER (ORDER BY montant ASC) AS score_m
campaigniq(# FROM rfm_raw),
campaigniq=# rfm_segments AS (SELECT , (score_r::text || score_f::text || score_m::text)
AS rfm_code,
campaigniq(# CASE
campaigniq(# WHEN score_r >= 4 AND score_f >= 4 AND score_m >= 4 THEN
'Champion'
campaigniq(# WHEN score_r >= 3 AND score_f >= 3 THEN 'Client fidèle'
campaigniq(# WHEN score_r >= 4 AND score_f <= 2 THEN 'Client récent'
```

```
campaigniq(# WHEN score_r <= 2 AND score_f >= 4 THEN 'A risque'  
campaigniq(# WHEN score_r = 1 THEN 'Client perdu'  
campaigniq(# ELSE 'Client ordinaire' END AS segment FROM rfm_scores)  
campaigniq-# SELECT segment,  
campaigniq-# COUNT () AS nb_clients,  
campaigniq-# ROUND(AVG(recence_jours)) AS recence_moy,  
campaigniq-# ROUND(AVG(frequence)) AS frequence_moy,  
campaigniq-# ROUND(AVG(montant):: numeric,0) AS montant_moy  
campaigniq-# FROM rfm_segments GROUP BY segment ORDER BY nb_clients DESC;
```

P3-T3 : Analyse RFM - Segmentation client (KAN-48)



[KAN-60] Sauvegarder la table RFM pour Power BI

Création: 21/mai/26 Mise à jour:

22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T3 : Analyse RFM - Segmentation client

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank:	0 i000aw:
--------------	-----------

Description

```
campaigniq=# CREATE OR REPLACE VIEW rfm_view AS
campaigniq=# WITH date_ref AS (SELECT MAX(invoice_date)::date AS ref FROM retail),
campaigniq=# rfm_raw AS(SELECT
campaigniq(# customer_id,
campaigniq(# (MAX(d.ref) - MAX(invoice_date)::date) AS recence_jours,
campaigniq(# COUNT(DISTINCT invoice) AS frequence,
campaigniq(# ROUND(SUM(total_price)::numeric,2) AS montant,
campaigniq(# MAX(country) AS country
campaigniq(# FROM retail, date_ref d GROUP BY customer_id),
campaigniq=# rfm_scores AS (SELECT *,
campaigniq(# NTILE(5) OVER (ORDER BY recence_jours DESC) AS score_r,
campaigniq(# NTILE(5) OVER (ORDER BY frequence ASC) AS score_f,
campaigniq(# NTILE(5) OVER (ORDER BY montant ASC) AS score_m
campaigniq(# FROM rfm_raw)
campaigniq=# SELECT *, (score_r::text || score_f::text || score_m::text) AS rfm_code,
campaigniq=# CASE
campaigniq=# WHEN score_r >= 4 AND score_f >= 4 AND score_m >= 4 THEN
'Champion'
campaigniq=# WHEN score_r >= 3 AND score_f >= 3 THEN 'Client fidèle'
```

```
campaigniq-# WHEN score_r >= 4 AND score_f <= 2 THEN 'Client récent'  
campaigniq(# WHEN score_r <= 2 AND score_f >= 4 THEN 'A risque'  
campaigniq-# WHEN score_r = 1 THEN 'Client perdu'  
campaigniq-# ELSE 'Client ordinaire' END AS segment FROM rfm_scores;
```

P3-T4 : Visualisation sur Power BI (KAN-49)

 **[KAN-61] Connecter Power BI à PostgreSQL** Création: 21/mai/26 Mise à jour: 22/mai/26 Résolu: 22/mai/26

État:	Terminé
Projet:	Workshop
Composants:	Aucun(e)
Versions affectées:	Aucun(e)
Versions corrigées:	Aucun(e)
Parent:	P3-T4 : Visualisation sur Power BI

Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		

Rank: 0|i000b4:



P3-T4 : Visualisation sur Power BI (KAN-49)			
 [KAN-62] <u>Crée les visuels - KPI cards</u> Création: 21/mai/26 Mise à jour: 22/mai/26 Résolu: 22/mai/26			
État:	Terminé		
Projet:	Workshop		
Composants:	Aucun(e)		
Versions affectées:	Aucun(e)		
Versions corrigées:	Aucun(e)		
Parent:	P3-T4 : Visualisation sur Power BI		
Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		
Rank:	0 i000bc:		

<u>P3-T4 : Visualisation sur Power BI</u> (KAN-49)			
 [KAN-63] <u>Graphiques et slicers</u> Création: 21/mai/26 Mise à jour: 22/mai/26 Résolu: 22/mai/26			
État:	Terminé		
Projet:	Workshop		
Composants:	Aucun(e)		
Versions affectées:	Aucun(e)		
Versions corrigées:	Aucun(e)		
Parent:	P3-T4 : Visualisation sur Power BI		
Type:	Subtask	Priorité:	Medium
Rapporteur:	Naia Landaboure	Personne assignée:	Naia Landaboure
Résolution:	Terminé	Votes:	0
Étiquettes:	Aucun(e)		
Estimation restante:	Non spécifié		
Temps consacré:	Non spécifié		
Estimation originale:	Non spécifié		
Rank:	0 i000bk:		

Généré à Fri May 22 14:01:12 UTC 2026 par Naia Landaboure avec JIRA 1001.0.0-SNAPSHOT#100291-rev:5829cedafba6a8fa42a0906c5ee4ff46e2db21de.